RISC-V Sodor Design Documentation

1. RISC-V Chisel code set for Sodor

1.1. Convention

Chisel code set for Sodor is assumed to be stored in an area, with variable $SODOR. This can be in either user area (/home/<username>/riscv-sodor) or a common area so that any user can view them, such as /projects/riscv-sodor/.

e.g. `export $SODOR=/home/<username>/riscv-soder`

Unless specified, the directories of Sodor design/verification contents are located below this path.

1.2. Sodor Microprocessor Family

Sodor is a family of rv32 microprocessor designs and currently includes a single stage, 2-stage, 3-stage, and 5-stage pipeline processors.

1.3. Chisel Code File Directories

Each design shares a common code set, which is in $SODOR/src/common area. Processor specific codes reside in $SODOR/src/rv_32_<pipeline#>stage sub directories. Also, ROM microcode for micro-instructions are separately stored in $SODOR/src/rv32_ucode. The Fig. below is Sodor processor family's Chisel code directory structure:

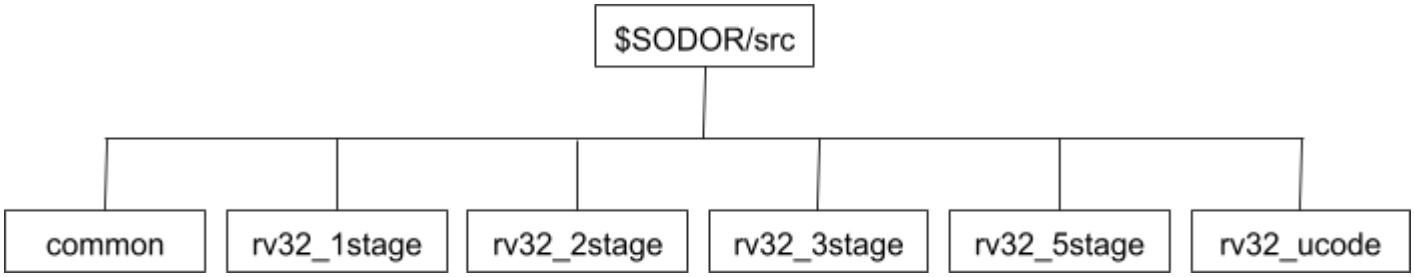

Fig. Sodor Chisel code directory structure

From the following sections, each of Sodor microprocessor design's block diagram, some more details of Chisel code & their structures are discussed.

1.4.Sodor 1-stage(rv32_1stage)

1.4.1.10,000ft view: Block Diagram

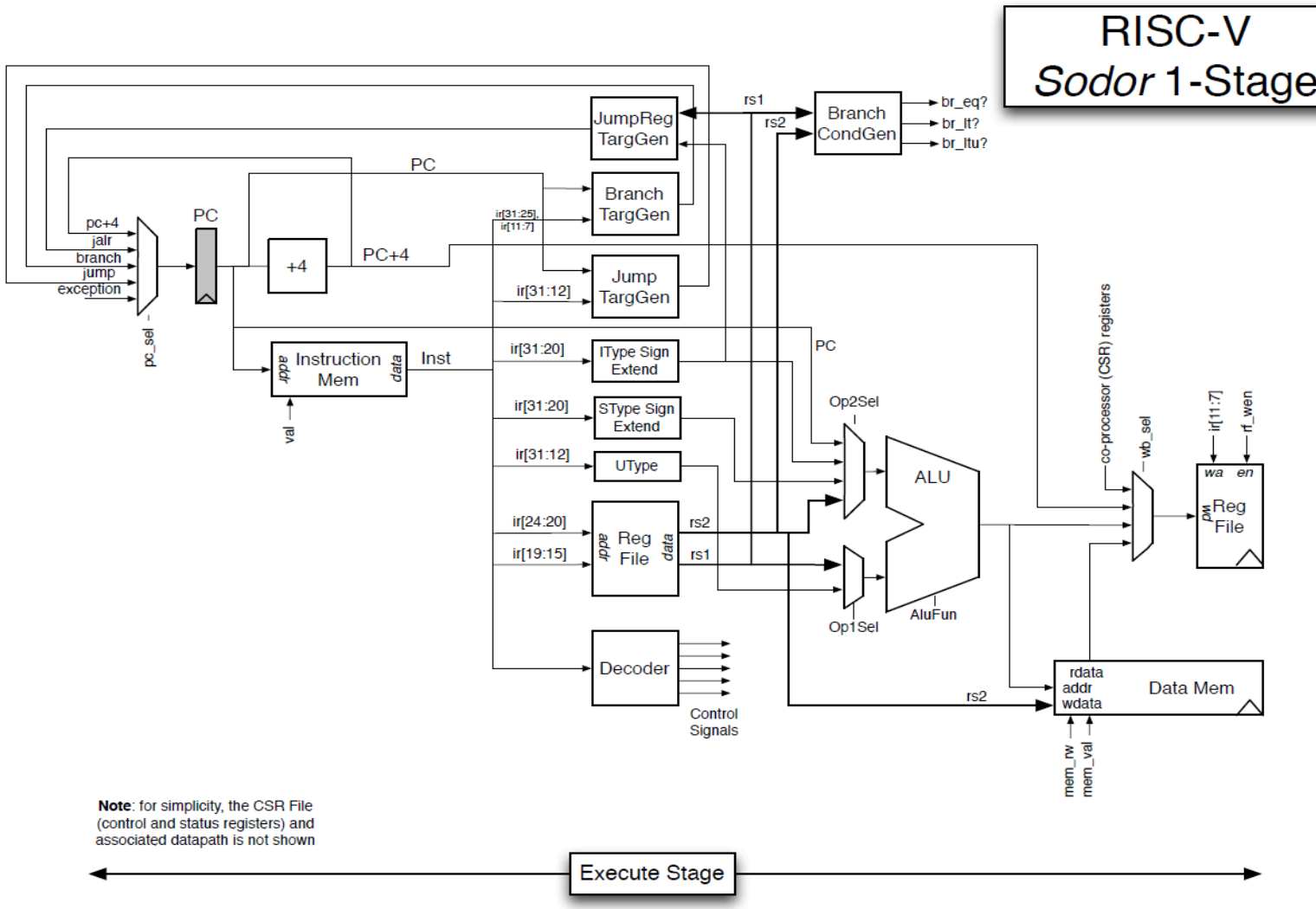

Fig.1. Sodor 1-stage processor block diagram.

1.4.2.Block descriptions

rv32_1stage is the foundation of the entire RISC-V microprocessors. This is a single stage microprocessor and essentially behaves identical as what RISC-V ISA simulator does.
Pipeline separators, flip flops are placed in front of program counter(PC) and instruction memory block, to drive out rv32's micro-instructions into register file for input source selections, instruction type selections, jump/branch controls, and so on. These controls are then sent to ALU, data memory, branch condition detector(Branch condGen), and/or program counter selector, which is fed into flip flop for the next program counter PC. Fig. 2. shows the key blocks in the processor and corresponding control and data paths.

This processor uses microcoded programs for each instructions, and scratchpad RAM is also used for boot up time(any other condition for use?). The microcode ROM and are connected to instruction Mem in the diagram(any other block connected?).

1.4.3.Design hierarchy: $SODOR/common

1.4.3.1. configurations.scala

This file contains variables for processor's architectural feature set; it includes xprlen, indicates 32-bit architecture(?), rvc, indicates if this processor uses compant instructions, if the processor use virtual mode(?), and usingUser(?); current settings are described below.

- xprlen = 32 << samte as XLEN; Sodor is 32-bit CPU and hence this is 32
- nxpr = 32…? << only defined in the file but never used in any other files; no longer needed
- nxprbits = log2ceil(nxpr)...? << only defined in the file but never used in any other files
- rvc = false << RISC-V compressed instruction feature
- vm = false << Virtual mode feature? only defined in the file but not used in any files
- uingUser = false…? looks like a condition if the operation is under user mode?

1.4.3.2. consts.scala

This file defines RISC-V instruction's bit positions for source/destination register, CSR address bit MSB/LSB, shift amount data field for shift instructions, and NOP(or BUBBLE). These constants defined in this file are also described in "[The RISC-V Instruction Set Manual Volume II: Privileged Architecture](Current: Draft v1.10 as of Nov. 6, 2017)." Please refer the doc for more details.The file also includes Privilege mode encoding and mode bit count(SZ_PRV), and address space definitions for physical, virtual, page index address bits, along with some more supporting bit definitions such as MTVEC(Machine Trap-Vector base address) and start address at boot up(reset?).

- X0 = 0.U …?
- MTVEC = 0x100 << Machine mode Trap VECtor base address
- PPN_BITS = PADDR_BITS-PIDX_BITS << Physical Page Number bit
- VPN_BITS = VADDR_BITS-PIDX_BITS << Virtual Page Number bit
- ASID_BITS = 7 << Address Space IDentifier
- PERM_BITS = 6 …?

1.4.3.3. csr.scala

This file defines CSR(Control and Status Register) bit encodings. CSRs are 32-bit register set for RV32 processors, or defined with architectural variable XLEN, and these registers hold various processor's ISA feature set, operation modes, interrupt, trap and exception conditions and states, performance monitoring related data registers, debug modes/states, etc.
All instructions are categorized into 7(X & N are the same?) types, defined at the beginning of object CSR, they are N(X), W, S, C, I, and R type. and they are 3-bit values and corresponds to a section of instruction encodings, bit 14 thru 12. These instructions are atomic instructions, and the sequence of atomic instructions are described in details in [The RISC-V Instruction Set Manual Volume I: User-Level ISA](Current: v2.2 as of Nov.8 2017).
Definitions of each CSR register are also explained in detail on "[The RISC-V Instruction Set Manual Volume II: Privileged Architecture](Current: Draft v1.10 as of Nov. 6, 2017)." Please refer the doc for more details. The definitions of all CSR contents are described in Chapter 2 through 7. Please refer to the document for the details.

The register is grouped into the following major class and objects, associating their interface/assignment classes:

- class Mstatus: Machine Status
- class DCSR: Debug Control and Status Register
- object PRV: Privilleged mode
- class MIP: Machine Interrupt Pending
- object CSR:

1.4.3.4. instructions.scala

This file is automatically generated by “parse-opcodes”(? description needed). The file describes all instruction encodings, with “?” bit fields designate the source/destination register and/or immediate values. The file also list the trap causes, and the entire list of CSRs(Control and Status Registers). the definitions of the CSRs are defined in csr.scala. There’s also codes(commented out) for disassembler which was replaced by riscv-dis.

1.4.3.5. memory.scala

This file defines synchronous/asynchronous scratchpad memory block used in rv32_?stage core designs. Only 3-stage core uses synchronous scratchpad memory while all other cores use asynchronous scratchpad memory. Current implementation does not detect write collision(hazard) as described in the header section.

Interface definitions and Asynchronous(Synchronous) scatchpad memory are defined as the following Figs:

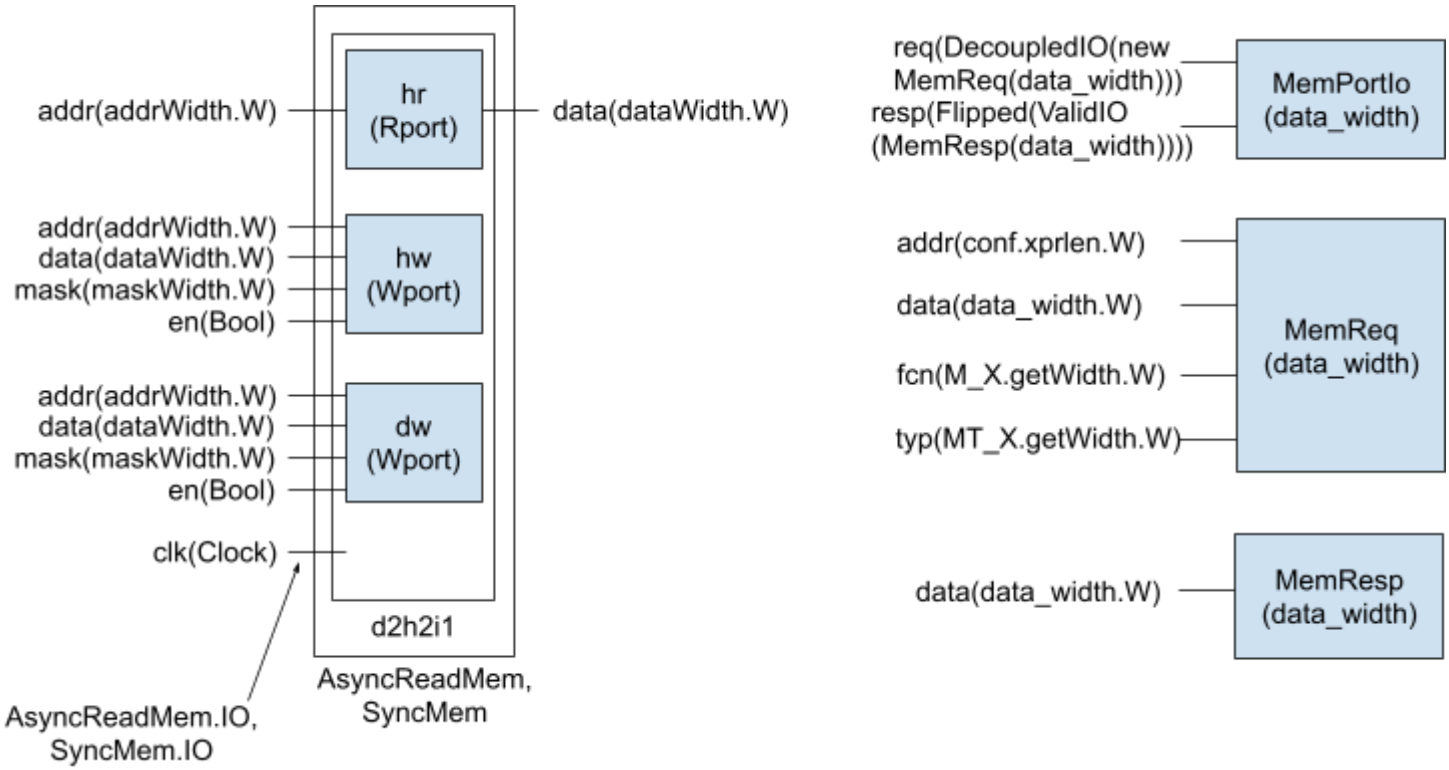

Fig. memory.scala interface definitions

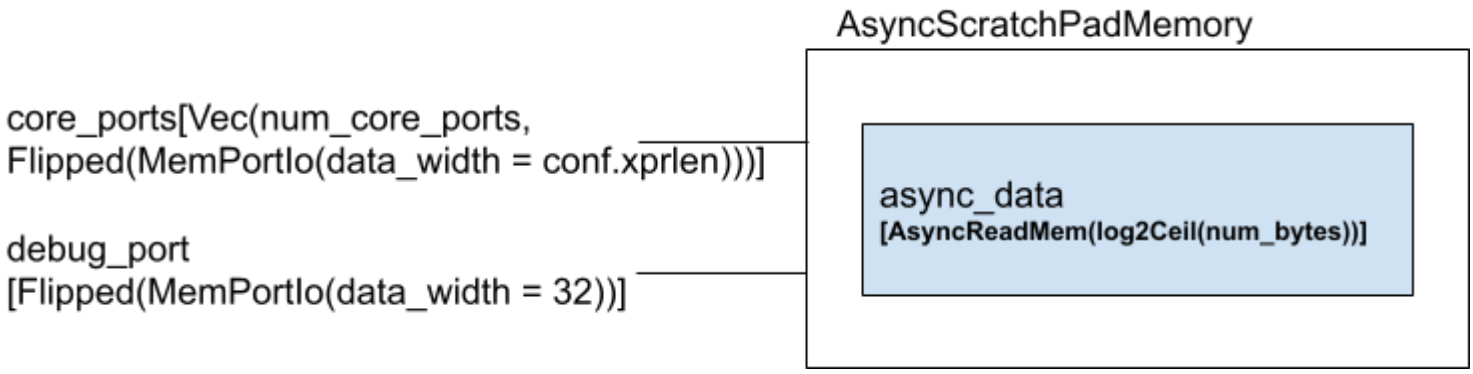

Fig. AsyncScratchPadMemory block diagram; SyncScratchPadMemory is very much the same except for a flop inserted to capture io.debug_port.resp.valid with io.debug_port.req.valid ANDed with condition if io.debug_port.req.bits.fcn matches to 'M_XRD' operation(1-bit match)
class 'ScratchPadMemory' is defined here but this is not used; replaced with AsyncScratchPadMemory.

1.4.3.6. debug.scala

RISC-V debug module. RISC-V Debug spec is currently under definition stage and the draft spec document is available(Version 0.13 as of Nov 9, 2017).

1.4.3.7. dm_registers.scala

The file is auto-generated from thru 'make chisel' command in https://github.com/sifive/riscv-debug-spec.git. This defines DMI(Debug Module Interface)
The details of Debug Mode is described in "RISC-V External Debug Support(Version 0.13, as of Nov. 6, 2017)"

1.4.3.8. abstract_commands.scala

The file is auto-generated from thru 'make chisel' command in https://github.com/sifive/riscv-debug-spec.git. This defines bit fields for class ACCESS_REGISTERFields(debug.scala calls this) and QUICK_ACCESSFields, and these are debug mode command and data(both in/out) designation and other definitions. Please review "RISC-V External Debug Support(Version 0.13, as of Nov. 6, 2017)"

1.4.3.9. package.scala

This file contains a definition object Constants as a workaround for a bug SI-5604; Please review comment "TODO" section.

1.4.3.10. util.scala

This file contains "Macro" definitions to ease codings by providing many type conversion methods and/or functions for all the designs. Some are defined but not used.

1.4.4.Design hierarchy: $SODOR/rv32_1stage

1.4.4.1. consts.scala

This file contain a mapping tables for each micro-operations within the core for the PC select MUX, Branch, source/destination selectors, ALU and memory access types to map each operation with UInt or bool type designation.

1.4.4.2. cpath.scala

This file defines entire instruction's internal operation controls as a list of control types for each control block(PC, Branch, rs1, rs2, rd, ALU op, mem access, CSR op) along with it's I/O definitions.
Also, all fo the instruction parameters described above are anumerated as three categolies, csignals, cs0, and cs1, to group them as memory access control, ALU op and its result's destination controls(include mem. access controls), and the entire control set.
The next PC pointer is defined as an outcome of all branch condition results as MUXes.
Stall conditions are defined as either imem or dmem are not ready(euqtion seems to be redundant...any reason?):
Exception conditions are defined as either A)Ileagal instruction and B)CtlPath.io.imem.resp.valid… but can't find the definition of B)...

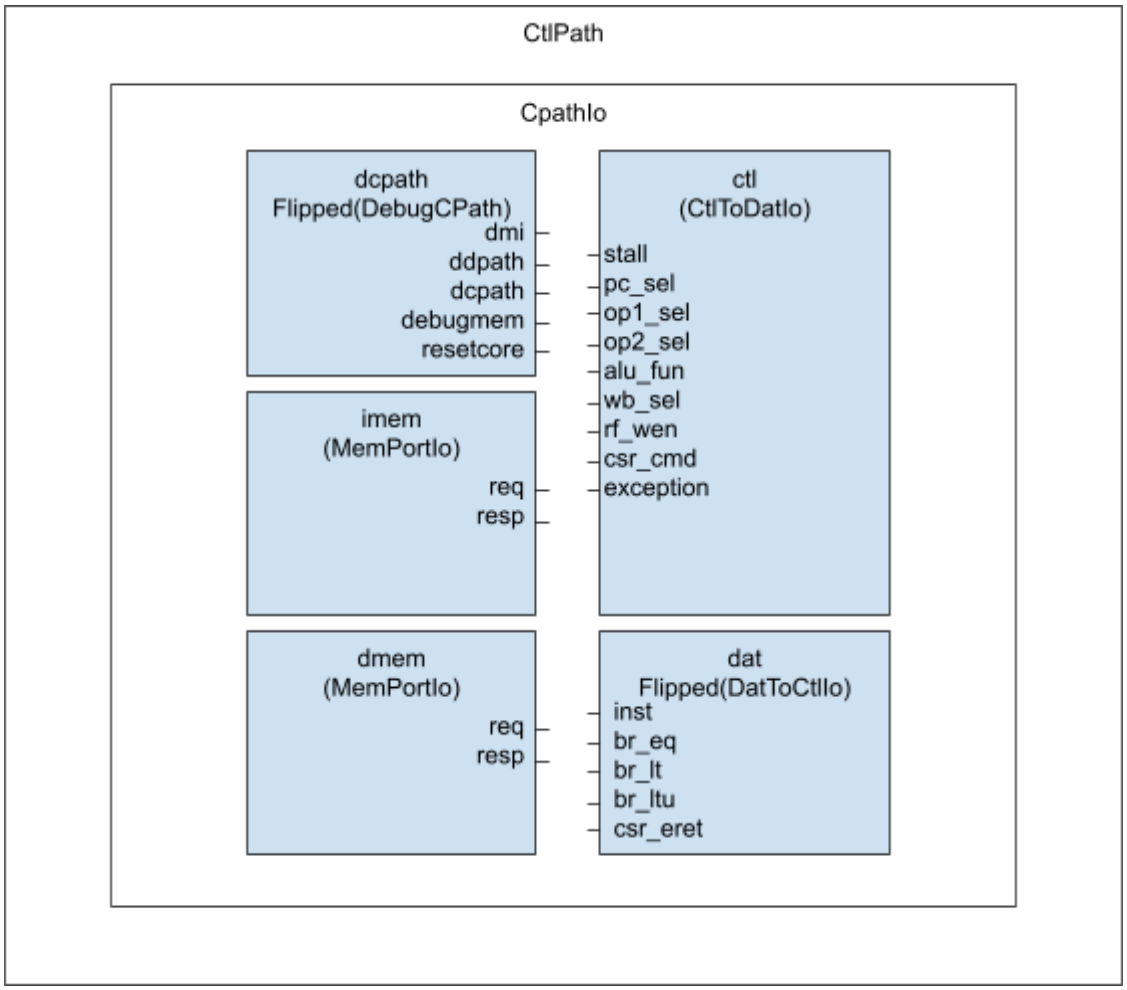

Fig. cpath.scala block diagram. Note the interface definitions at CpathIo and CtlPath is not defined as they are implicitly defined within blocks at subblocks in CpathIo, and some of them are bundle of control and/or data signals. Imem and dmem are defined in common/memory.scala

1.4.4.3. dpath.scala

The file describes PC control definitions as to how PC for the next instructions will be based on branch control signals from cpath.scala conditions. Source/destination registers and immediate values in each instruction are also decoded. ALU operands controls for each instructions are defined here as well. Some CSRs are updated based on the CSR command, stall info, exceptions.

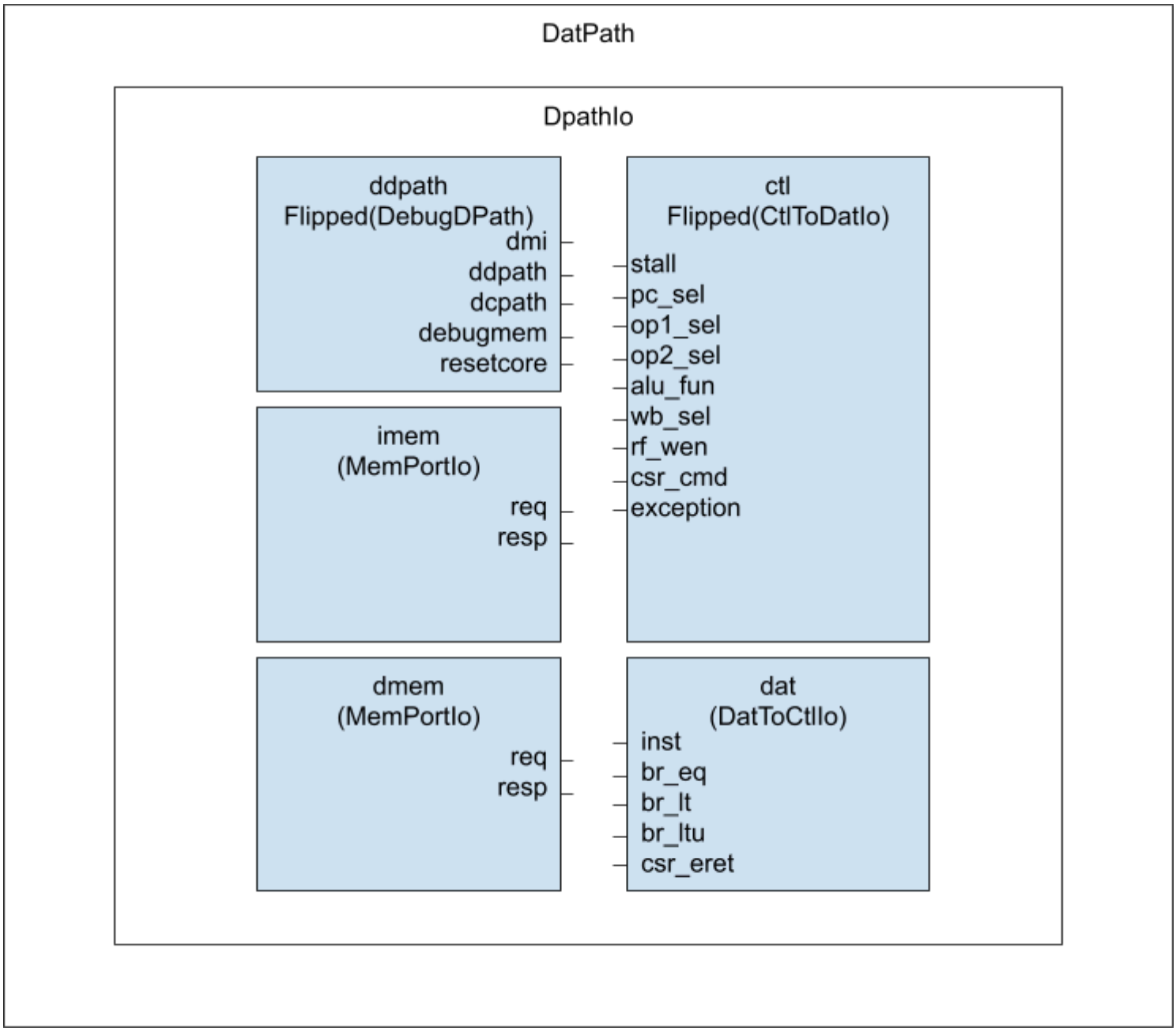

Fig. dpath.scala block diagram. Note the similarity between cpath.scala, except for I/I direction by ‘Flipped’ notation. Control definitions/descriptions are in place accross the hierarchy and/or blocks.

1.4.4.4. core.scala

This file describes the processor's core block instantiations and their control/data signal connections. The blocks are instantiated as c(CtlPath), d(DatPath), and IO(CoreIo) as showin in Fi.g below:

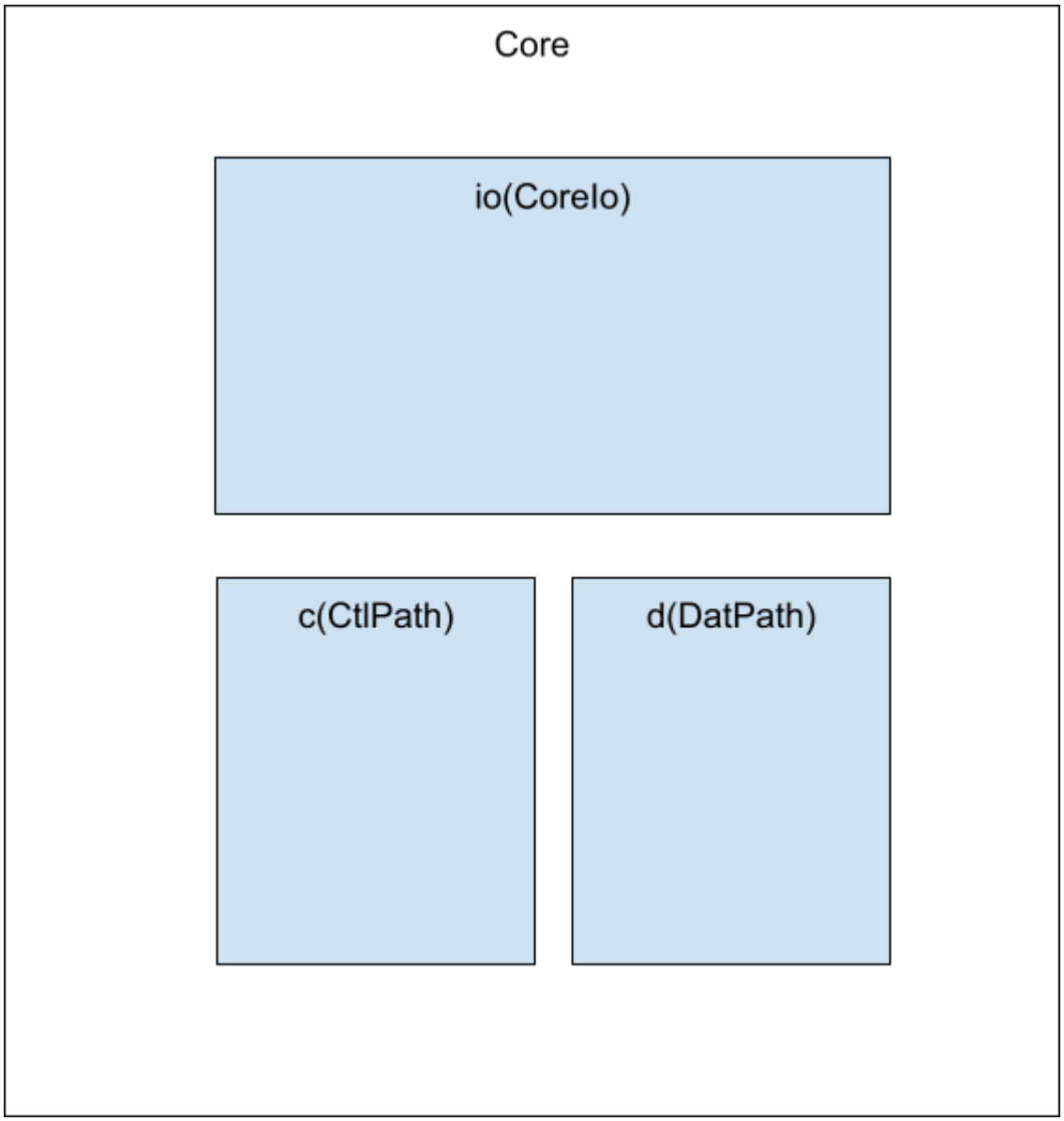

Fig. core.scala diagram. Only instantiating its subblocks and some remaining connections between the blocks are described in the file for the processor core definition.

1.4.4.5. tile.scala

The file defines the entire Sodor ‘tile’ blocks and their bus/data/control connectivities. The block diagram at this level is in the following Fig.

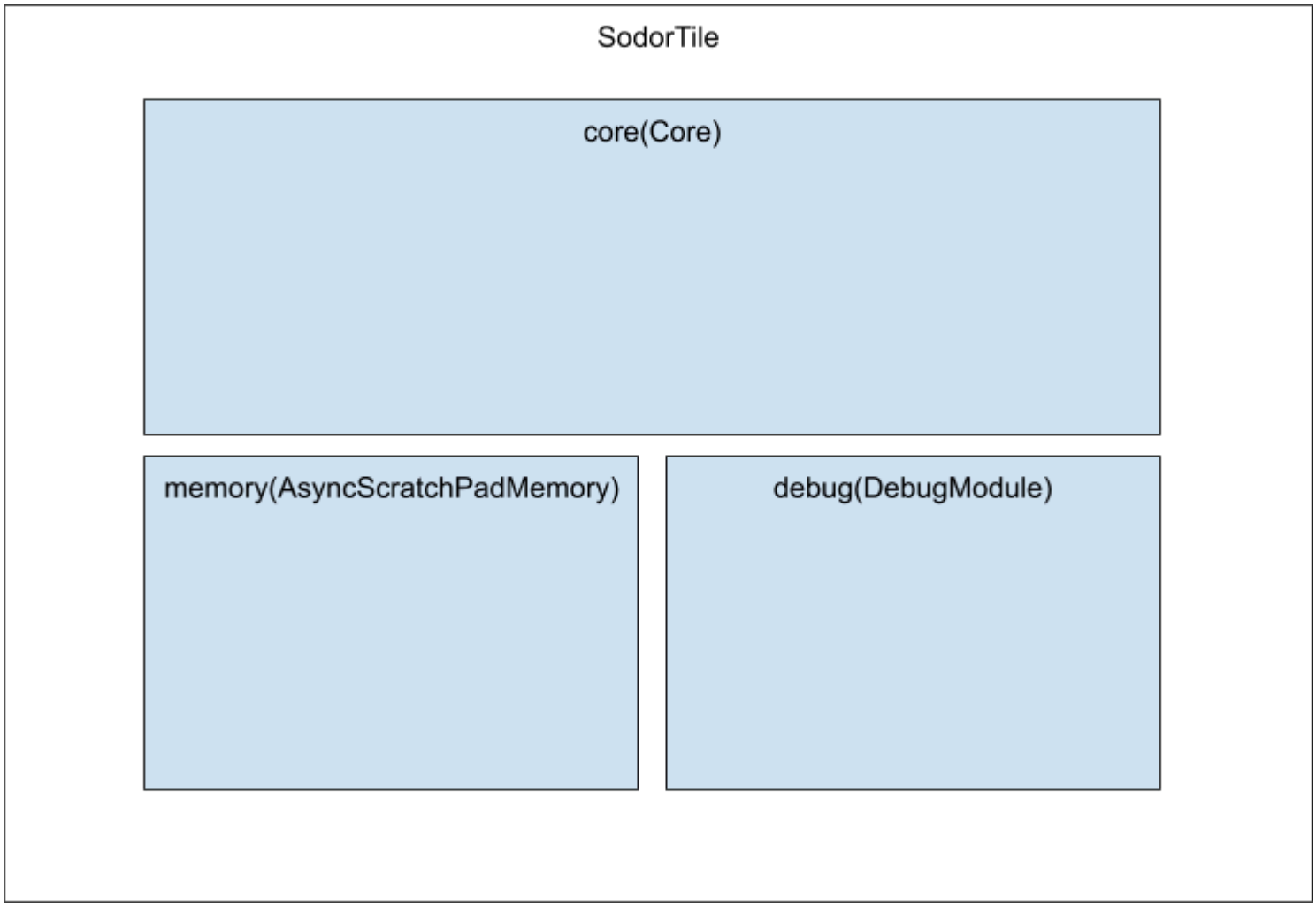

Fig. SodorTile diagram. This corresponds to the microprocessor top level.

1.4.4.6. Top.scala

The file is a wrapper to include DTM(Debug Test Module) defined as SimDTM and SodorTile, which will interface with SodorTile for debug. The block diagram is shown at the top half of Sodor benchmark & Validation System Block Diagram in Section 1.4.6.

1.4.4.7. package.scala

This file is the same purpose as the common/package.scala described in 1.3.4.9. For the rest of the design.

1.4.5.Design hierarchy: $SODOR/rv32_ucode - This is a micro-coded processor

1.4.5.1. consts.scala

This file maps micro-instruction's control, source/destination registers, write, ALU operations, immediate control, branch type code, micro-PC state logic definitions into UInt values.

1.4.5.2. microcode.scala & micrcompiler.scala

microcode.scala lists the micro instruction sequences for each RV32 instruction. The micro-operations used in consts.scala are used. Each instructions are described by explaining micro-operation per line, starting from what micro-operation the microcode will issue, and which micro-instruction variables are used, from the const.scala, and if micro-branch(UBr) will be taken or not, what ALU operation will be done, which register will be used and where the results will be stored back.

The entire micro-operation table is described in the table **here**:

Microcodecompiler.scala creates actual microcode ROM bitmap, and generate "opcodeDispatchTable." The compiler will read microcode.scala and parse labels & Signals(contains all microinstruction control signals) which specify what control variables are used and use const.scala values to encode the microcode ROM bits for each entry.

1.4.5.3. cpath.scala

This file calls MicrocodeCompiler object in microcodecompiler.scala and its uOP dispatch table is used for generating control signals to dpath,imem and dmem blocks shown in Fig. on section 1.4.4.2. This also defines u-operation PC(upc), as well as micro-PC state register.

1.4.5.4. dpath.scala

The file takes cpath.scala file's control signals for data source(bus) selection, loading source data (both registers/immeidiate), immediate value extractions from instruction code bits(high order bits are filled to complete XLEN data width). 32 XLEN registers are also defined in this file. Also, the file defines how CSR is accessed just like a regular registers for read and write, and also defines how CSR handles exception, updating evec and eret values, as well as updating PC.

ALU operations are aldo defined in the file.

1.4.5.5. core.scala

This is very much the same as rv32_1stage core.scala. The conly differences are the separation of scratchpath memory for instruction & data on rv32_1stage while rv32_ucode use a single scratchpad memory.

1.4.5.6. tile.scala

This is also very similar to rv32_1stage tile.scala except for memory block seperation between instruciton and data on rv32_1stage.

1.4.5.7. top.scala

This file is very much the same as rv32_1stage.

1.4.6.Sodor benchmark & Validation System Block Diagram

TBD

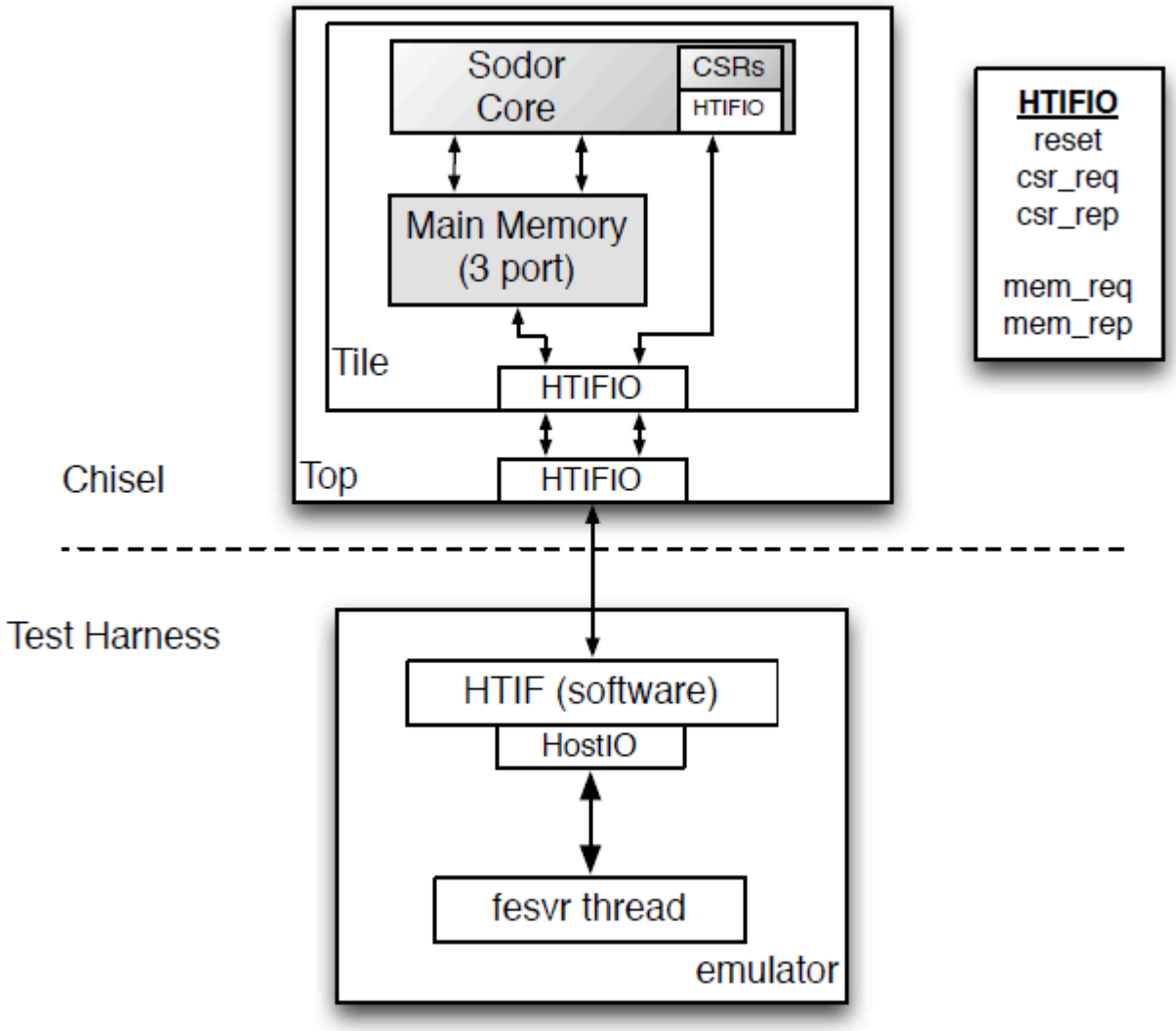

Fig.2. Sodor validation & debug system block diagram. Note Main Memory(3-port) corresponds to the ScratchPadMemory in the design code as in tile.scala.

1.4.7.

1.4.8.

1.5.Sodor 2-stage(rv32_2stage)

1.5.1.Block Diagram

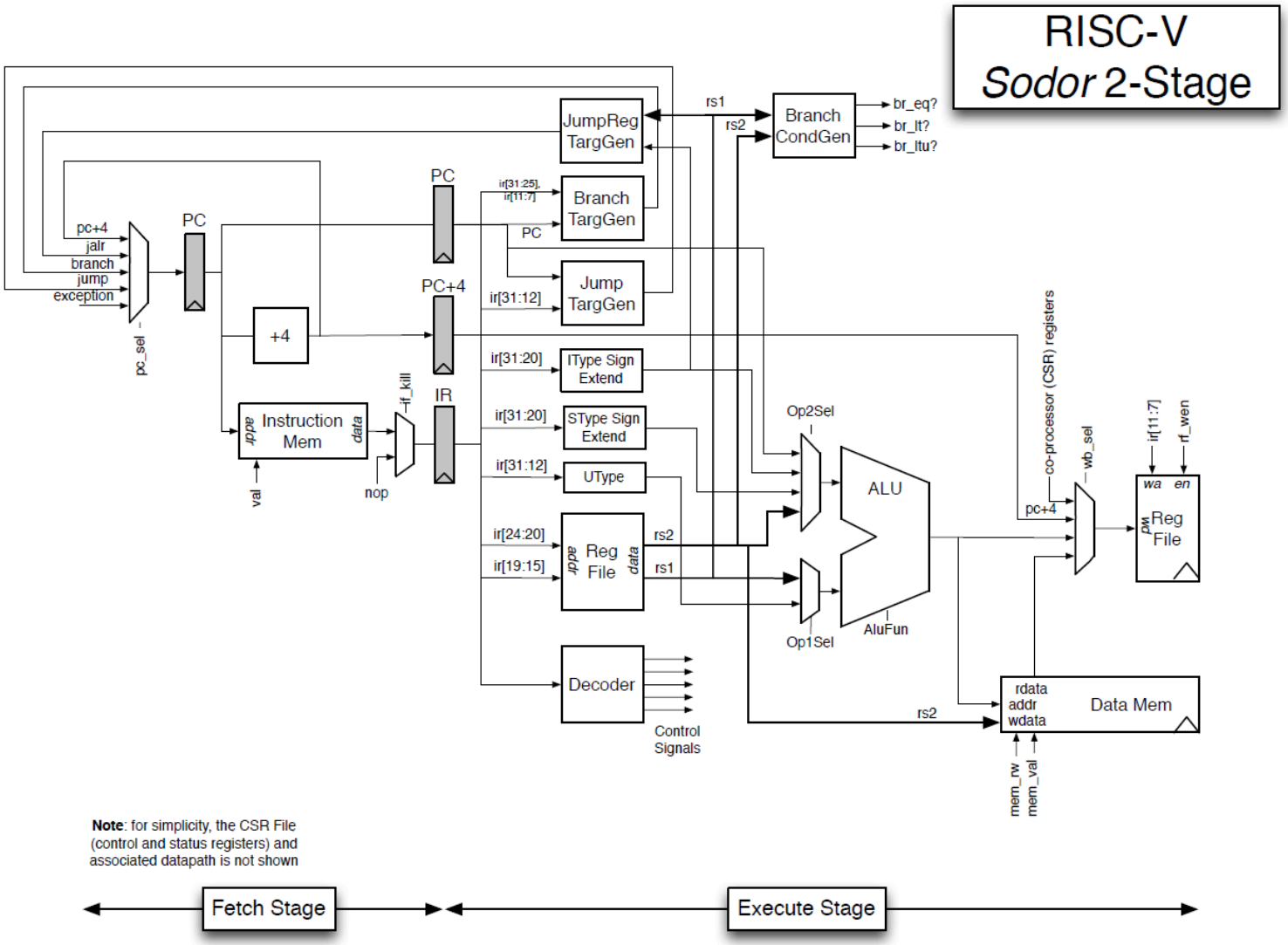

Fig. Sodor 2-stage processor block diagram

1.5.2. Block descriptions

This 2-stage processor add an extra pipestage by seperating decode & execution, and instruction fetch, inserting flops after program counter(PC) and instruction memory access, just before instructino decode stage, as shown block diagram on 1.5.1. This will make the pipeline as

A) Instructions will be read from instruction mem, based on PC pointer
B) Decode and execute the instruction

The remaining controls & connections are maintained very much the same w.r.t. Rv32_1stage design.

1.5.3.Design hierrarchy: $SODOR/rv32_2stage

1.5.3.1. consts.scala

There are few differences from rv32_1stage design; A)writeback address selects are added, although they are not used anywhere in the design. Also, B)memory(cache) flushes & sync. And read/write/fence memory functions are removed.

(Reason: TBD)

1.5.3.2. cpath.scala

The differences from rv32_1stage are addition of ‘if_kill’ Bool value. If instruction memory is in the process of read, or the execution stage is still in a process(it defines if I$ or D$ misses), ‘if_kill’ will be asserted and wait for one more clock, by inserting ‘nop’ into execution stage for the next cycle as shown on the block diagram Fig. in section 1.5.1.

1.5.3.3. dpath.scala

The new additions to this file from rv32_1stage are A)additional flops for the PC feeding into executions stages, called
exe_reg_pc
exe_reg_pc_plus4
exe_reg_inst
which corresponds to the flops in Fig. in section 1.5.1., and MUX to select instructions from I$, or a bubble(Nop) selected by if_kill signal. Also all the inputs for execution stages are renames to take outputs from new PC stage flops described above. Corresponding input signal names are renamed with ‘exe_’ or ‘ or if_’ prefix, depending on the signals used for execution blocks, or execution PC controls.

1.5.3.4. tile.scala

There is no difference from rv32_1stage design.

1.5.3.5. Top.scala

The only one difference from rv32_1stage is the library usage at the beginning of the file is removed, and corresponding codes which calls the lib is removed. (Reason? TBD)
Lib. removed: chisel3.iotesters.{ChiselFlatSpec, Driver, PeekPokeTester}

1.5.3.6. package.scala: identical as rv32_1stage

1.5.4.

1.6. Sodor 3-stage(rv32_3stage)

1.6.1.Block Diagram

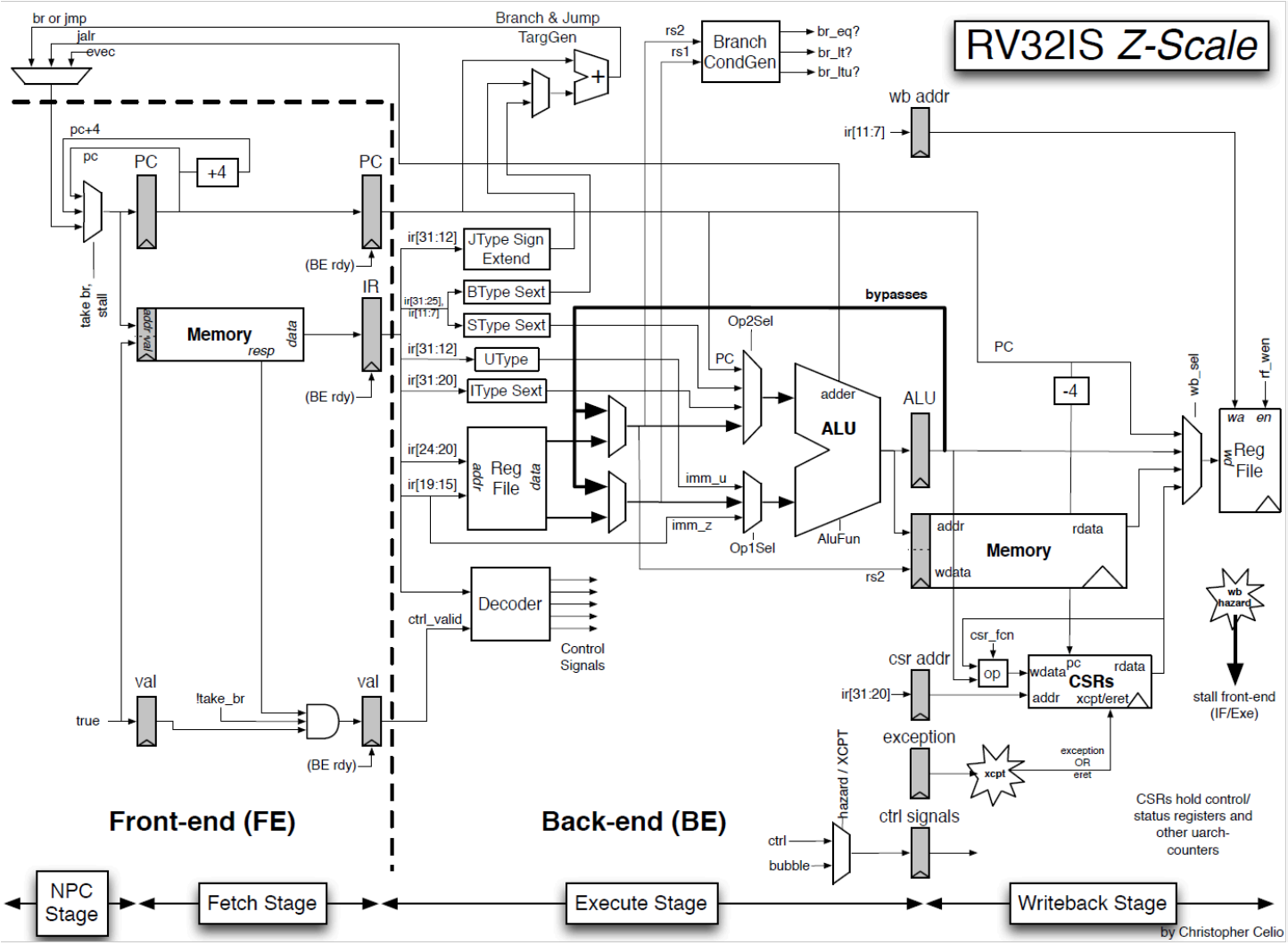

Fig. Sodor 3-stage(rv32_3stage, Z-Scale)

1.6.2.Block descriptions

Rv32_3stage design is a significant departure from rv32_2stage, although it added just one pipestage to rv32_2stage; ALU outputs are flopped, and routed back into both or either side of its input for next cycle(MUXed with register file outputs). D$ starts getting accessed in write back stage. CSR updates are also allocated as the same stage as the register file write back. In addition, any branch address inputs into PC flops are moved out from PC select MUX, and a separate branch address MUX stage is inserted whthin execution stage. Note that because the register file is accessed in both execution & write back stage, and I$/D$ are sharing the same scratchpad memory, there could be hazards between the access requests to the register file and/or I$/D$, and in such case, either or both fetch and execution stage could stall. There are some more additional flops such as exception and CSR control signals(address, commands), in order to accommodate appropriate pipeline states.

The design files are reorganized, and the following 3 new files are added; arbiter.scala is to arbitrate the scratchpad memory, which uses single-port memory(this is only 3-stage design):

A) frontend.scala
B) alu.scala
C) arbiter.scala

1.6.3.Design hierarchy: $SODOR/rv32_3stage

1.6.3.1. arbiter.scala

This file defines an FSM to alternate access to I$ and D$ request with d_fire signal, which enables D$ data request to be driven onto data bus from memory.

1.6.3.2. frontend.scala

The file very much describes what's drawn in the block diagram; state value of 'if_reg_valid' will decide if the next PC should be PC+4. As a default it's set always true. Read the description of the header section for possible modification(enhancement) as FYI. The block also defines I$ request procedure, response to the requests from execution block(a flop gated with "BE rdy" in diagram), and debug controls.

1.6.3.3. alu.scala

The file defines all ALU functions, and its interface, as well as ALU operation encodings.

1.6.3.4. const.scala

Since this design uses single port memory as scratchpad memory(for instruction and data), and also ALU components are defined in a separate file(alu.scala), the definitions for memory and ALU bit encodings are removed and moved into corresponding files. Everything else is the same as rv32_2stage:const.scala.

1.6.3.5. cpath.scala

cpath.scala's CtlToDatIo() is changed to CtrlSignals() and includes D$ controls (valid, function & type), bypass (to bypass instruction results), and execution kill(exe_kill) to trash execution stage in case of exception/mret conditions. "Bypassable" is a condition defined within instruction list table in the file, and this includes load, store, branch and CSR control, and debug/fence commands, which does not use ALU(true?).

1.6.3.6. dpath.scala

There are new registers added to the file for staging ALU outputs, valid bit(for exe. stage outputs?), register file write back address, CSR write command & address, as well as staged versions of cpath CtrlSignals. Also a new wb_hazard_stall signal is added to detect write back hazard condition, specifically checking read after write hazard.

In the file, the write back stage's PC address is 4-byte less than 'exe_pc'(execution stage's PC) is because write back stage's pipestage is one instruction before what execution stage uses, hence one instruction(which is 32-bit or 4-byte) worth of address is subtracted from exe_pc.

1.6.3.7. core.scala

This hierarchy defines as I/O, and 3 components, and the components are frontend, control path(cpath) and data path(dpath). The I$ and D$ (imem/dmem) are actually on I/O and hence there's a definition for connecting them between frontend and dpath, as well as cpath.

1.6.3.8. tile.scala

Since this 3-stage design place scratchpad memory with parameter "NUM_MEMORY_PORTS" (1 single port memory array) to share both I$ & D$, the connections between imem and dmem connections with a single port memory case is added. Also, arbiter is called in this file to hook up to memory.

1.6.3.9. top.scala: identical to 2-stage design

1.6.3.10. package.scala: similar to 2-stage design; some extra

1.7.rv32_5stage

1.7.1. Block Diagram

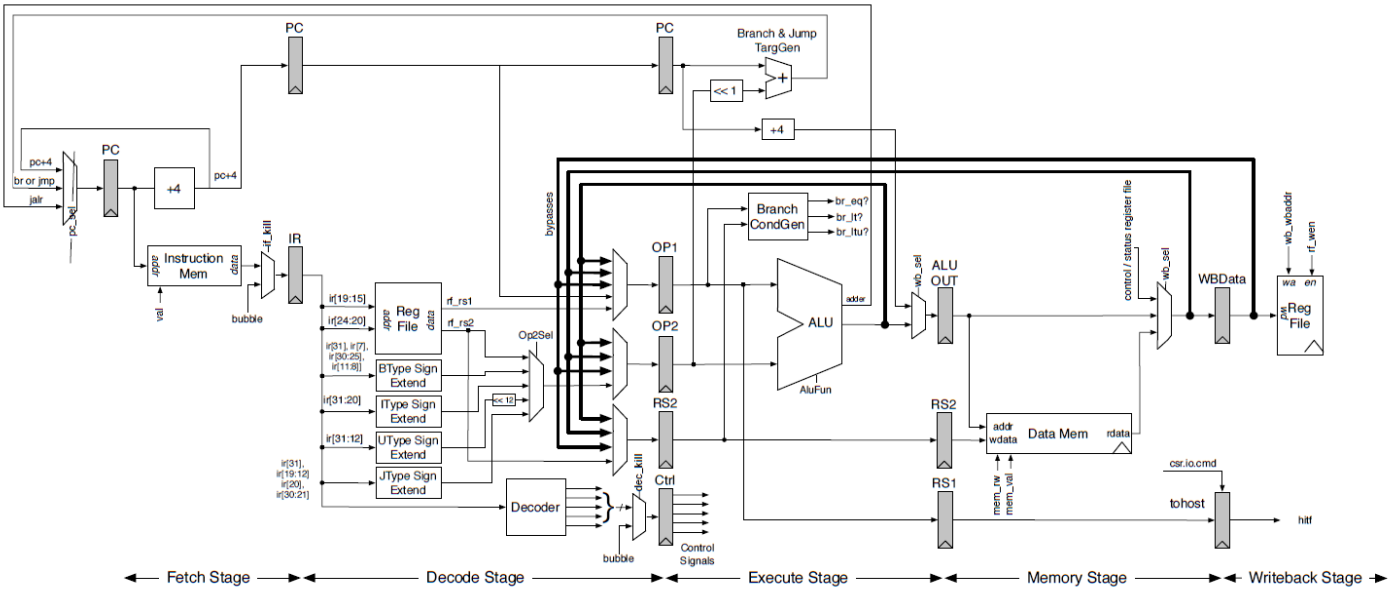

Fig.3. rv32_5stage block diagram

1.7.2. Brief Description of the design

The 5-stage design is an extension from 3-stage design, by dividing 3-stage design's execution stage into decode & execute stages, and also dividing writeback stage into memory and register file accesses. Pipestages are names as

IF: Instruction Fetch

DEC: Instruction Decode

EXE: Execution

MEM: Memory Access

WB: Write Back to register file

Each specific stage's signals has prefix, such as if_, dec_, exe_, mem_, and wb_ accordingly.

1.7.3. Design Hierarchy: $SODOR/rv32_5stage

1.7.3.1. const.scala

The design assume full-bypassing, which means the machine only stalls when load-use condition(?). A new design variable "USE_FULL_BYPASSING" is set true.

PC select variables are reduced to 4(was 5 in 3-stage design); hence encoding bit count is also reduced from 3 to 2. Also, the source of two ALU inputs, operand1 & operand2 are re-arranged; in 3-stage design, **operand1** is one of A)RS1, B)unsigned immediate, C)zero-extended immediate and D)unassigned(X) and 5-stage designs takes one of **A)RS1, B)PC, C)zero-extended immediate or D)unassigned(X)**. For **operand2**, 3-stage design takes A)RS2, B)Integer immediate, C)Store type immediate, D)PC, and E)unassigned(X). In 5-stage design, it takes either **A)RS2, B)Integer type immediate, C)Store type immediate, D)branch type immediate, E)unsigned type immediate, F)jump type immediate, and G)unassigned(X)**.

ALU operations are also defined in 5-stage design.

Memory write operation encoding is simplified with just one bit(write or not-write). Memory mask type definition is the same as 3-stage design.

1.7.3.2. regfile.scala

This is a new file for 5-stage design. It defines a register size of 32-entry(RISC-V definition), XLEN data width. It defines whether input data should come from regular write operation, or debug mode write operation. This is because depending on these condition, write data input path will be different.
It is not explicitly described, but the register file is 3-port array so that both rs1, rs2 and debug port can be read out. When read is happening the write cannot occur.

1.7.3.3. cpath.scala

5-stage design has two stall signals; one for IF/DEC stage stall, and the entire pipeline stall. Also, corresponding kill signals to trash IF or DEC stage, and another kill signal for all IF/DEC/EXE stages in case of an exception, detected in memory stage, are added as well. In addition, an indicator signal for executing FENCE.I instruction is also added.
Instruction Lookup list for this design adds **r1/r2 output enables and fencei indication bit(Bool)**, but removed 'bypassable' entry. WFI instruction is implemented as NOP(refer inline comment) in this design.
Source register output enable(rs1_oen/rs2_oen) are gated by deckill signal; in case deckill is asserted output enable will be deasserted(does this make any difference other than power?).
In the design, stall condition is defined as the following two cases(see inline comments); a)I$ and/or D$ access conflicts, and b)I$ miss. When neither of the case, the machine will check if decode stage has to be killed(meaning the instruction is branch/jump, the current PC is not PC+4), and if it is the case the machine will hold write back into register file(which means the machine has to read I$ for the next instruction). If decode stage doesn't have to be killed, then write back to the register file, CSR state update, and exception flag(condition signal) will be updated accordingly. In case of stall, write back will be always put on hold and instruction read will proceed.
Stall conditions are exactly coded as described above, with a condition of "USE_FULL_BYPASSING" variable; the default 5-stage design sets this variable as true, and hence the hazard conditions are very few w.r.t. When the variable is set to 'false.'

The hazard condition is defined as all exe, mem, wb stage's writeback address (*_reg_wbaddr) matches to decode stage's source register addresses for either of rs1/rs2, then the stall signal is set, in case of not using full bypassing; in that case is should be only decode and wb stage's condition should be checked…

(Question)The code states as the following:

```
// stall for load-use hazard
      stall := ((exe_inst_is_load) && (exe_reg_wbaddr === dec_rs1_addr) &&
                                (exe_reg_wbaddr != 0.U) && dec_rs1_oen) ||
                ((exe_inst_is_load) && (exe_reg_wbaddr === dec_rs2_addr) &&
                                (exe_reg_wbaddr != 0.U) && dec_rs2_oen) ||
                (exe_reg_is_csr)
```

Here, in the first term, exe_inst_is_load refers there will be a memory read in the next cycle; exe_reg_wbaddr refers write address for register file write to rs1 for the 2nd cycle, after the current cycle, and dec_rs1_oen corresponds to the output gating for driving out rs1 into the execution stage for the next cycle… none of them seems to be leading hazard or at least nothing wrong with the operation, which seems to be just reading memory in the next cycle, then writing back some data into register file in the following cycle(just looking at operations throughout pipeline stages).

Full stall condition is defined as D$ miss; here's the code:

```
full_stall := !io.imem.resp.valid || !((dmem_val && io.dmem.resp.valid) || !dmem_val)
```

Here, dmem_val corresponds to io.dat.mem_ctrl_dmem_val(dpath.scala), and this is mem_reg_ctrl_mem_val (dpath.scala). And this is a flopped version of from exe_reg_ctrl_mem_val (dpath.scala). This is another flopped version of io.ctl.mem_val(cpath.scala), and it's one of decoded outputs of instruction, cs_mem_en(cpath.scala).

1.7.3.4. dpath.scala

DatToCtlIo definition now include "mem_ctrl_dmem_val" which indicate memory access operation is valid. Many decoded instruction's control signals from decode stage is flopped twice, for execution and memory stages, and writeback operation controls(wraddr, wrdata, wen) are also flopped 3 stages. There are quite a few control signals to serve stall conditions to insert NOP and flush the pipeline, depending on which stage needs to be flushed.

CSR are also revised accordingly; retire signal is set if both io.ctl.full_stall and io.ctl.dec_stall are not true… this doesn't seem to indicate the instruction can really retire. The condition should rather be wb_reg_valid...which is replaced with the current code...from 3 stage design.(?)

1.7.3.5. core.scala: similar to 3_stage

1.7.3.6. tile.scala: similar to 3_stage

1.7.3.7. top scala: similar to 3_stage

1.7.3.8. package.scala: similar to 3_stage

1.7.3.9.